Operating System Security

Week 5: Authentication

Authentication and Identity

Identification is stating who you are (e.g., entering your username).

Authentication is “proving you are who you claim to be”.

- Authentication proves your identity.

Why does authentication matter for operating systems security?

Background: Passwords Are Stored 'Hashed'

Alice's password= Security1

Login Request:
Username=Alice,
password= Security1

Recommended algorithm for H:
- scrypt

Not recommended:
- MD5
- SHA*

Server

H(Security1) = *A125EF89E1F9326*

User	Hashed Password
Alice	*A125EF89E1F9326*
Bob	*1F9831A2E5B4A73*
...	

Threats to Passwords: Online Guessing Attack

If the system does not limit login attempts, attacker could perform an online attack to guess passwords.

- Password guesses are verified by the system itself.

Attacker could be typing the passwords into the system, or using a script (more likely)

Threats to Passwords: Online Attack

Wordlist
password1
password123
…
Security1

Attacker's Login Request:
Username=Alice,
password= password123

Login failure

Login success!

Server

H(password1) = *59E1FA899984230F*
H(password123) = *2BEEFE001975EF8*
H(Security1) = *A125EF89E1F9326*

User	Hashed Password
Alice	*A125EF89E1F9326*
Bob	*1F9831A2E5B4A73*
…	

Threats to Passwords: Offline Guessing Attack

If hashed passwords stolen, attacker could perform an offline attack to guess passwords

- Password guesses can be verified on adversary's computer.

Consider how hashed passwords can be stolen…

Threats to Passwords: Offline Attack

Attacker's Computer

H(password1) =
91FE19EA121F32695EF84683733751305440E6C0
…
H(Security1) =
A125EF89E1F9834AC57A749BEC430E6BCFF09326

Wordlist
password1
password123
…
Security1

User	Hashed Password
Alice	*A125EF89E1F9834 AC57A749BEC430E 6BCFF09326*
Bob	*1F983A125EF9BEC 430E6BC89E4AC4F F093257A73*
…	

Methods to Generate Guesses (1/2)

Brute-force

- All possible passwords are guessed
- Consider how inefficient this would be!

Markov-model-based

- Guesses are generated based on common language patterns (but the guesses may not represent words).
- E.g., the sequences of 3 characters {t, h, e}, {e, a, c}, and {c,a,r} are common, so the markov-model based method would generate "theacar".
- Some popular password guessing programs have a mode that does this.

Methods to Generate Guesses (2/2)

Dictionary attack: guesses from a list of highly probable guesses.

- This list was historically a *dictionary*, but now it's most often a *wordlist*.

Some research results [1] show that if an attacker has an optimal dictionary, they can find: 4.4-15.9% of passwords in 1000 attempts, and almost 50% in approx. 1 million attempts. Targeted attacks can do even better [2].

Dictionary
password
password1
…
amethyst
angora
…
zebra123

[1] J. Bonneau, "The Science of Guessing: Analyzing an Anonymized Corpus of 70 Million Passwords", IEEE Symposium on Privacy and Security, 2012.
[2] D. Wang et al. "Targeted Online Password Guessing: An Underestimated Threat", ACM CCS, 2016.

How to Obtain an Optimal Dictionary?

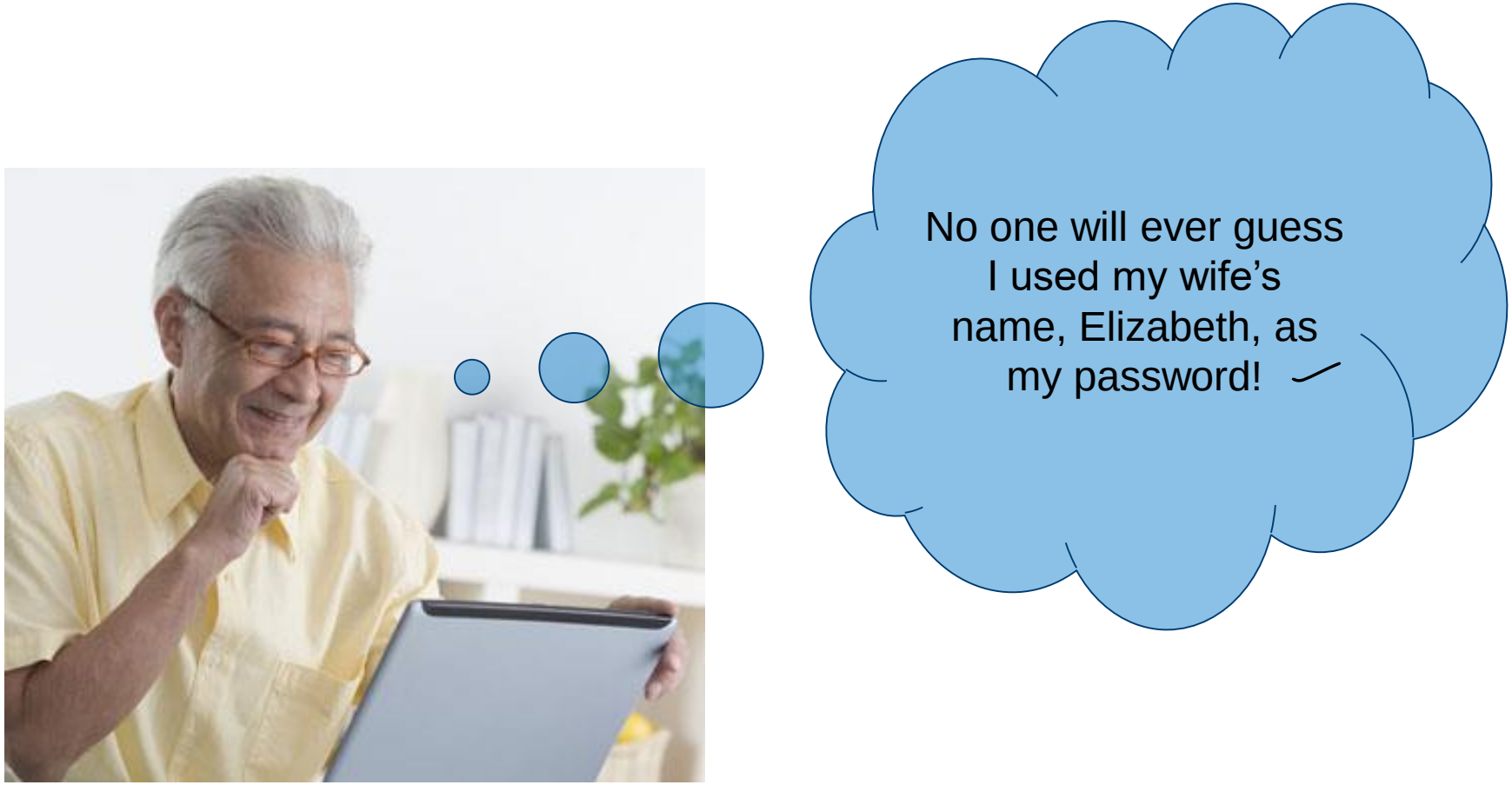
The results on the last slide assumed the adversary knows exactly what the most popular password are.

However, the attacker does not know this, and analysis of leaked passwords shows that it differs between sites.

However, advances are being made in approximating/modeling password choice.

- Human factors are an important factor…

Human Factors in Authentication



Distribution of Passwords

Not uniform -- people are not likely to choose z57k90jS as their password.

People have to remember their passwords.

- People tend to remember things that are “more concrete”, such as words that have meaning.

9-24 digits of md5	# of occurrences
49ba59abbe56e057	1030
965eb72c92a549dd	350
323b453885f5181f	204
8ad9902aecba32e2	188
…	…

Take a guess, what’re those passwords?

Password Cracking Programs

Many are freely available online.

Examples: John the Ripper, Cain and Abel, OMEN, RainbowCrack, Weir's probabilistic context free grammars (PCFGs), Semantic PCFGs (from Ontario Tech!), and Neural Network Guessers.

RainbowCrack uses "Rainbow Tables"

- Rainbow tables use pre-computation and a time-space tradeoff.
- This is one of the largest threats to passwords if the system does not 'salt' passwords (we will discuss later)

Time-Space Trade-off

Theoretically, if you had an infinite amount of hard disk space, what would be the fastest way to guess password hashes?

Infinite space is not really possible though (at least for most people), but some space could be used.

Rainbow tables perform some pre-computation of hashes, in a way that makes them quickly searchable.

Password	H(password)
aaaaaaaaa	ad178409362….
Ilovedan	1d32as6782b…
alice123	45629a3b782c…
w0nderful	901abe23984…
12345678	f2e12o34ao2…
lalalala	e682a2fbc13…
…	…

Rainbow Tables: Hash Chains

R H

A rainbow table is comprised of a set of hash chains.

The chains are comprised of passwords and hash outputs.

The password p1 is hashed = H(p1) R(H(p1)) = p2

The hash h is used as input to a reduction function R, which deterministically outputs an element in a specific "password space" (e.g., set of all lowercase 8-character passwords), i.e., R(H(p1)) = p2

…

R

p1 → H(p1) → p2 → H(p2) → … → pn

Rainbow Tables

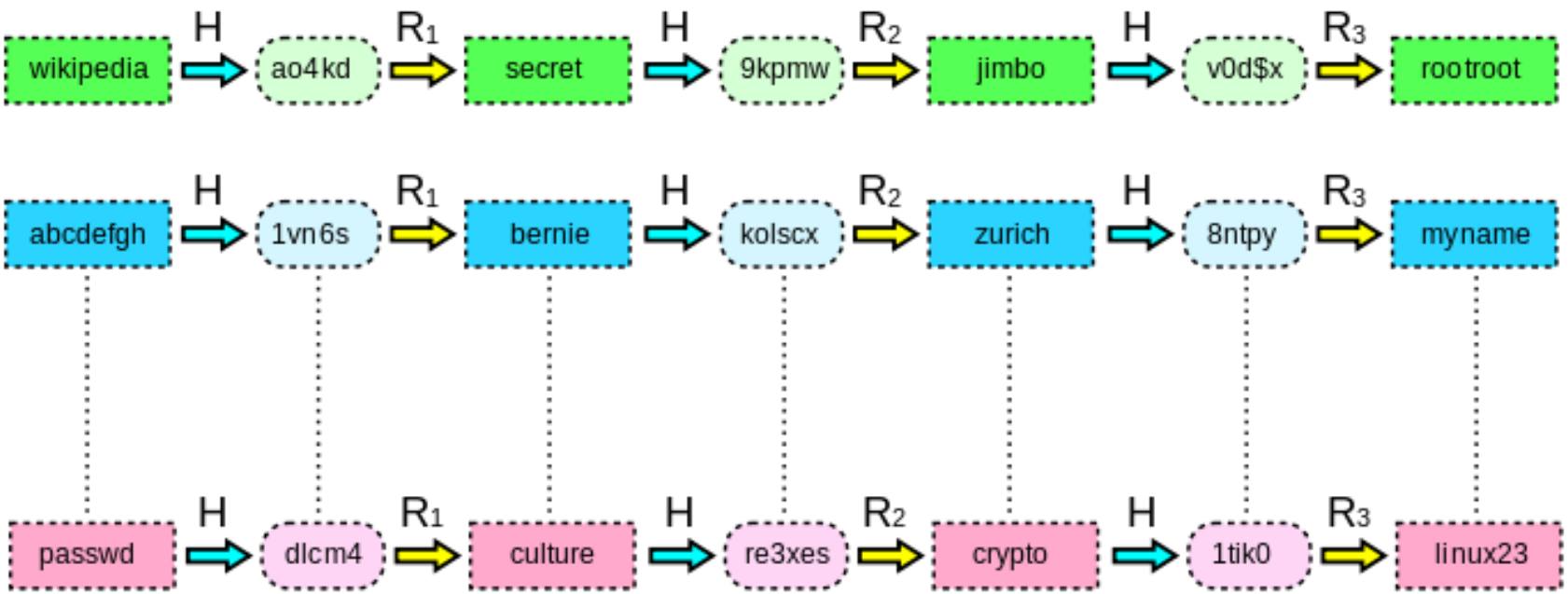


*Image originally from [*], courtesy of Dake.

For better performance, rainbow tables use more than one reduction function.

[*] Dake, “Rainbow Tables”, http://en.wikipedia.org/wiki/File:Rainbow_table1.svg , 2006, last accessed Jan. 2020.

Using Hash Chains

Many chains are computed to create a table.

The chains are compressed for storage: only the first and last element of each chain is stored.

- Given a hash h', try to form a chain using the reduction functions until you find a matching password in the last column of the table.
- Then take the start of the chain and re-compute it (keeping the last password).
- Once you find the matching hash, you know the password in the chain beforehand is the correct one!

Using Rainbow Tables to Guess a Hash

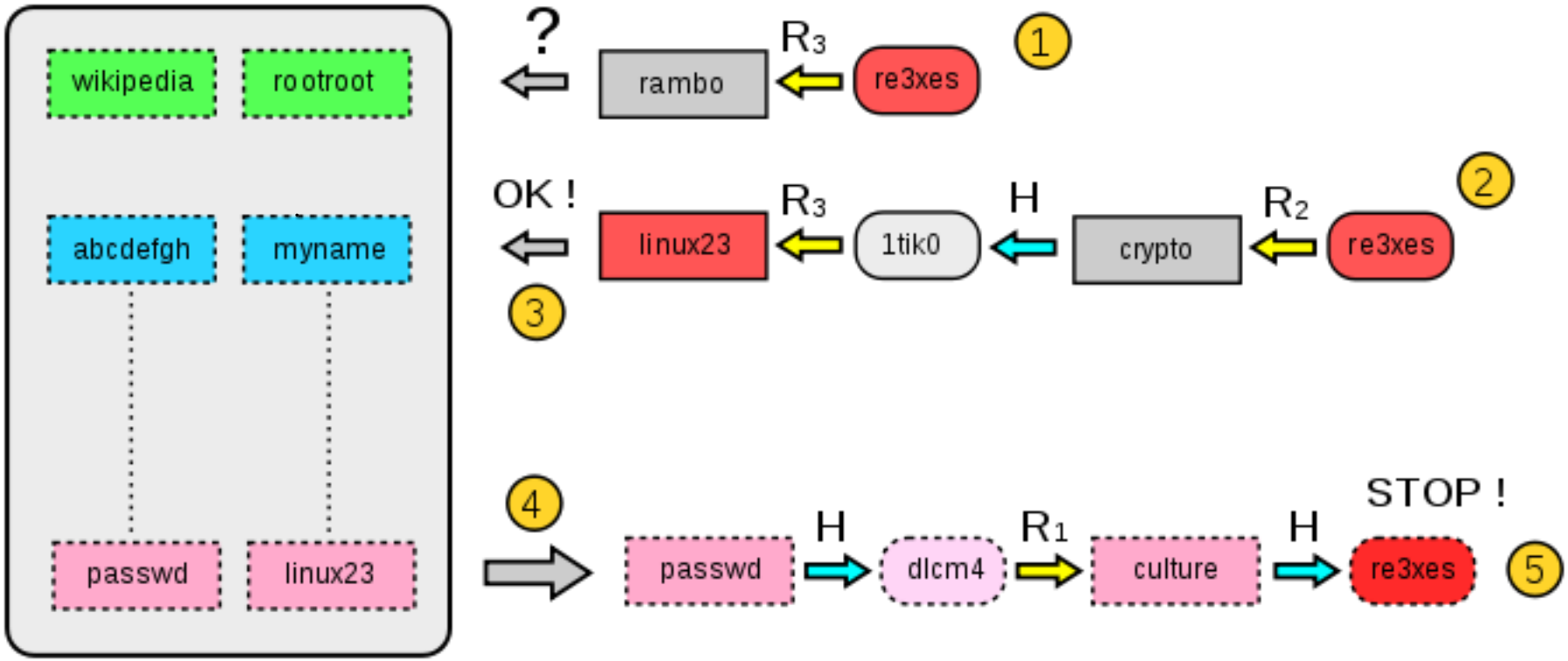


*Image originally from [**], courtesy of Dake.

[**] Dake, “Rainbow Tables”, http://en.wikipedia.org/wiki/File:Rainbow_table2.svg , 2006, accessed Feb. 2018.

Context: Current Rainbow Tables

Some sites sell pre-computed hash tables

Consider the tables available for sale, and what this means for password security.

- If the largest table includes all 1-9 character passwords (with all character types), what passwords does it make sense to permit in a password policy?

Summary Questions

Despite the presence of authentication techniques with arguably stronger security and usability, passwords remain the most popular technique.

- Why do you think this is?

At some point, you will likely hear an argument that an offline attack is not an important threat, because if an adversary could access the password file, they must have root access and thus can also modify the user's password temporarily.

- Can you think of a counter-argument as to why offline attacks still matter?

Defenses for Offline Attacks: Salting

Appending a random t-bit string to password before applying hash function.

Alters the "flavour" of the password.

Both hashed password and salt are stored in password file.

Increases difficulty of guessing passwords against all users (not for directed attacks).

Defenses for Offline Attacks: Salting

Without salting

a
abc
apple
…
foo

md5('a')
md5('abc')
…
md5('foo')

Alice	md5('foo')
Bob	md5('abc')
…	…

With salting

a
abc
apple
…
foo

md5('a'||r_1)
md5('abc'|| r_1)
…
md5('foo'|| r_1)

md5('a'||r_2)
md5('abc'|| r_2)
…
md5('foo'|| r_2)

Alice	md5('foo'||r_1)	r_1
Bob	md5('abc'||r_2)	r_2
…	…	…

Discussion question

Consider why salting protects against rainbow tables – the attacker would need to create a rainbow table for each salt value!

Let's say a distribution of an OS used salting, but always started its first account's salt value = 1, and then auto-incremented from there.

Would these salts (auto-incremented, starting at value = 1) still prevent rainbow table attacks?

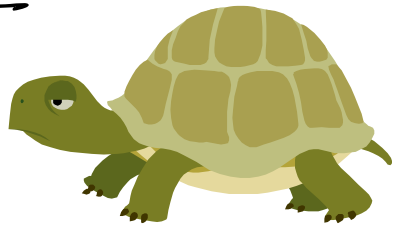
Salts: Important Implementation Factors

- They must be large (consider: what if you only had 16 possible salt values?).
- They must be randomly assigned!

Defenses for Offline Attacks: Slowing Down the Hash Function

The hashing function used for storing passwords can be made more computationally expensive.

- E.g. iterating the hash function t > 1 times.
- Means attacker needs to compute the hash function t times for each candidate guess.
- Value of t: trade-off between security and computation time that user will observe.
- E.g., bcrypt does this.



What are the pros and cons of this method?

Defenses for Online Attacks (1/2)

Account lock-outs

- After some number of incorrect login attempts from userA, prevent any more logins to userA's account.
- Disadvantage: many service requests to reset locked-out accounts.

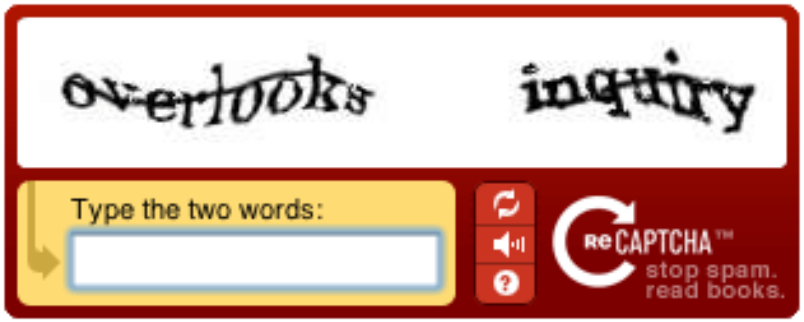
Add a login delay

- Upon incorrect password entry, delay the server's response (and thus the time before another guess).
- Disadvantage: attacker can still interleave password guesses between users.

Defenses for Online Attacks (2/2)

Solve a CAPTCHA every N failed login attempts

- Completely Automated Public Turing test to tell Computers and Humans Apart.
- Prove that you are a human and not an attacker's program.
- Use hard AI problems such as optical character recognition.



* Image from http://www.captcha.net/, last accessed Jan. 2020.

Some General Defenses for Guessing

Passphrases

- User types a phrase or sentence rather than a word. Goal is to increase strength.

Password rules

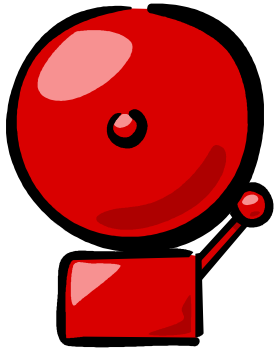
- E.g., “choose a password with at least one number, one upper-case character, and one special character”.
- These rules however are often added to password cracking programs.
- Many password rules used today are considered “out of date”.

Password strength meters

- Tells the user how strong their password is, based on some proactive checking metrics.

General Defenses for Guessing: Proactive Checking

When a password is created, the system checks whether the password is strong enough.



- Most often, this just involves checking complexity metrics, e.g., "does it contain >= 8 characters, one digit, and one capital letter?"
- Sometimes, it involves checking if it belongs to a wordlist. If it belongs, reject the password.
- Challenge: it must be fast to be done at the time the user is setting password.

General Defenses for Guessing: Reactive Checking

After password has been set, the system tries to actively guess the password using more advanced techniques.

If a password can be guessed, the system then forces the user to reset their password.

Advantage: this allows for security measures against more advanced password guessing attacks.

General Defenses for Guessing: System-Generated Passwords

System creates and assigns a “random” password to each user.

This is ideal for security against guessing attacks; however, what about usability?

- How would most users cope with this?
- Would you be happy if you were told your new password is “**QvMgy()e0-3Q^** ” ?

Some research focuses on improving memorability for system-assigned passwords.

General Defenses for Guessing: Password Aging

Common recommendation: passwords must be reset periodically (e.g., every 6 months).

Purpose: limit an attacker`s access to an account, in case they found the password.

Research [**] has shown that people tend to base new passwords on previous passwords.

- Also shows that if the previous password is known, that the next password can be guessed fairly quickly.
- Now controversial!

[**] Y. Zhang, F. Monrose, and M. K. Reiter. “The security of modern password expiration: an algorithmic framework and empirical analysis”, ACM CCS, 2010.

Important OS recommendations for passwords

- Salt with large random values.
- Set a password policy that requires long (and complex) passwords.
  - Can use rainbow tables available as a guide to what is computationally possible, and thus a sensible policy.
- Ideally, do proactive password checking (and reactive).
- Restrict repetitive login attempts (by lockout or delay).
- Keep the password hashes protected.

OS recommendations related to accounts

Disable an account if:

- The user will be gone for some time
- The accounts are seldom needed.

Some ways to prevent logins to an account:

- Change the account's login shell:

```
-Oscar:fdfdi3k1j$:105:100:oscar:/home/os
car:/etc/nologin
```

- Change it's password to something only you know.

(continued)

Set a dormancy threshold on an account with the –f option to the usermod command:

```
# usermod -f 10 foo
```

- In this example, user foo will have his account locked if a login is not made in 10 days

Set expiration date when creating an account:

```
# usermod -e 12/31/2007 foo
```